Blockchain y Criptomonedas II

UCEMA - QUANt

Clase II: 02/11/26

Matias Selser

Agenda

- Módulo V: Librería asyncio para Python
- Módulo VI: Discusión de la tarea propuesta
- Break
- Modulo VII: Repaso de mercados centralizados
- Modulo VIII: API de Binance
- Módulo IX: Librería CCXT
- Anexo: docker compose
- Conclusión y resumen
- Tarea propuesta

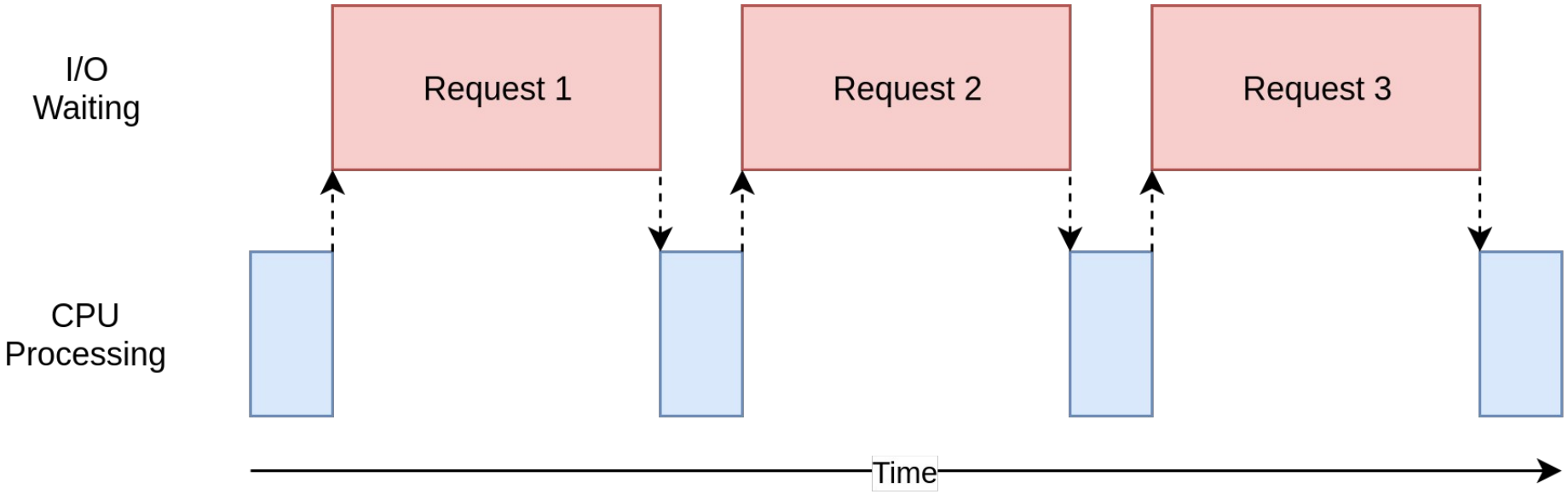
Módulo V
Librería asyncio para Python

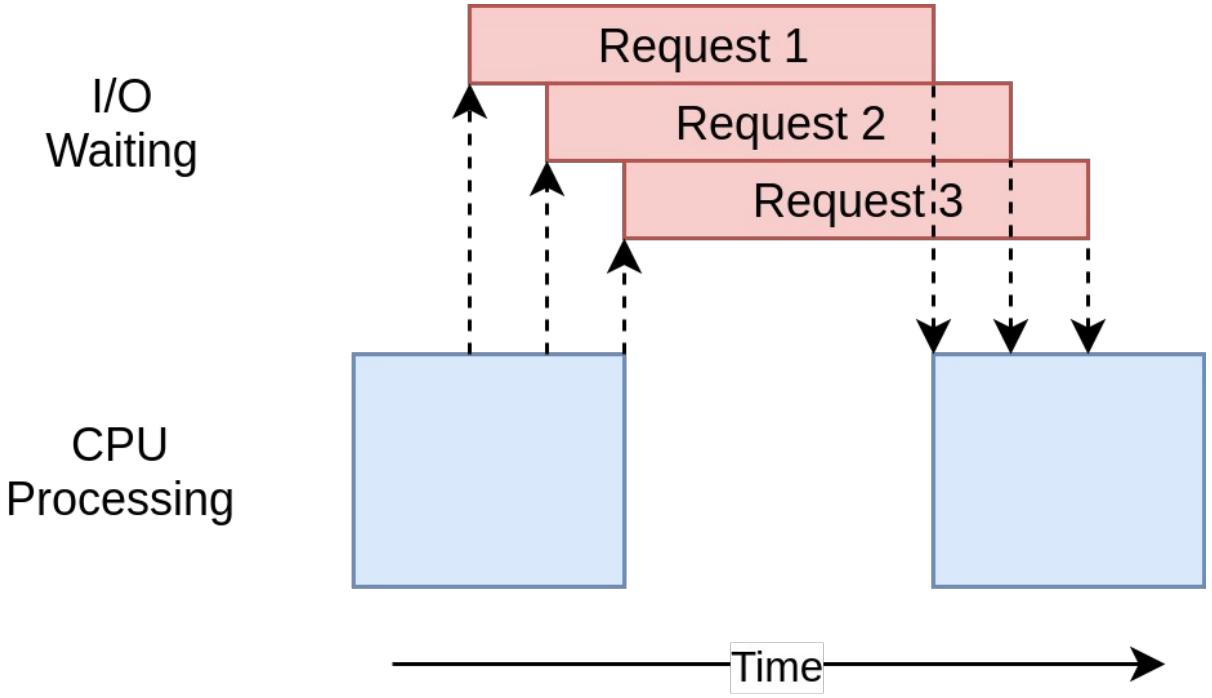
5.1 ¿Qué es asyncio?

- Librería de Python que permite escribir código **concurrente**
- Código concurrente:
  - Permite comenzar una operación antes de que finalice la anterior!
  - Se busca que las tareas se dividen en sub tareas que se puedan ejecutar en simultáneo.
  - Muy utilizado para:
    - I/O
    - Network requests

- Ejemplo: conectarse a n exchanges para obtener información de precios instantánea
  - Caso sincronico:
    - Request al exchange 1
    - Response del exchange 1
    - …
    - Request al exchange n
    - Response del exchange n
  - Caso asincrónico:
    - Request al exchange 1, 2, …, n
    - Respuesta del exchange 1, 2, …, n
  - Comparación

4.2 Comparación de performance




I/O
Waiting
Request 1
Request 2
Request 3
CPU
Processing
Time

Módulo VI
Discusión de las tarea propuesta

Break

Módulo VII
Repaso de mercados centralizados

7.1 Mercados centralizados

- ¿Qué son?
  - Son mercados con reglas más parecidas a los mercados financieros tradicionales
- Características
  - Funcionan con libros de órdenes
  - Se puede operar las 24 horas del día
  - Permiten operar:
    - *Spot*: settlement instantáneo, de monedas. Similar al FX
    - *Margin*: permite operar tomando margen prestado del mercado. Se colateraliza la posición con otra moneda o con la cartera
    - *Derivatives*: tienen una sección de derivados donde se pueden encontrar generalmente:
      - Futuros con delivery en *token*
      - Futuros con delivery en *stablecoin*
      - Futuros perpetual, sin delivery

- Perpetuals
  - Son futuros a perpetuidad, sin vencimiento
  - Siguen a un índice. Por ejemplo, *ETH/USDT*.
  - Tienen un *funding rate* asociado. El cálculo de este funding rate depende de cada protocolo
    - Busca hacer converger el precio del perpetual y el precio del índice
    - Por ejemplo: si muchos usuarios están *long* en el perpetual, el precio va a tender a subir probablemente por encima del índice. Eso genera un valor de funding rate tal que los usuarios que están *long* le van a pagar a los usuarios que están *short*. Por otro lado, el funding rate incentiva a los usuarios a tomar la posición contraria para recibir esta tasa.
- ¿Ventajas y diferencias del *margin* y los futuros?
- ¿Ideas de trading con esto?
  - ¿Futuros con delivery?
  - ¿*Perpetuals*?

7.2 Comparación entre *Dexes* y *Cexes*

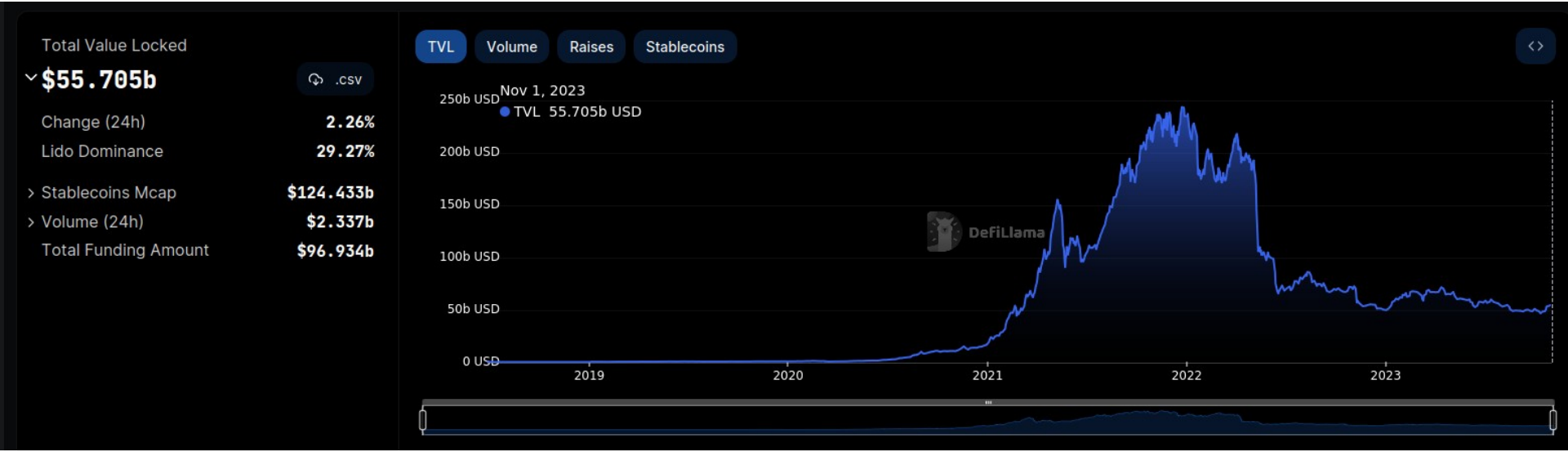
	Centralized Exchanges	Decentralized Exchanges
Privacidad	KYC	P2P
Seguridad	Discutible en ambos casos. Historial de hackeos y estafas en ambos lados	
Liquidez	Mayor liquidez, menos variedad	Menor liquidez, más variedad
Fees	Más caros	Más baratos
Velocidad	Muchísimo más rápido	<1 s (L2s) a 12 s (ETH)
UX	Más amigable para aquellos acostumbrados al trading tradicional	Hoy en día, suficientemente amigable para un usuario convencional
Trading	Contra el exchange	“Contra otro usuario”
Marco regulatorio"	“Sujeto a regulaciones”	No regulado, a veces auditado

Fuente: post

- Defillama.com
  - Cexes:

Name	Assets	Clean Assets	24h Inflows	7d Inflows	1m Inflows	Custom range Inflows	Auditor	Last audit date	Spot Volume
1 Binance	$65.971b	$57.908b	$95.38m	$695.28m	-$142.33m	-$16.43m			$7.629b
2 OKX	$11.297b	$11.297b	$24.16m	-$59.53m	$620.32m	$562.86m			$2.305b
3 Bitfinex	$10.811b	$8.268b	-$75.36m	-$77.06m	$24.45m	-$14.41m			$139.45m
4 Bybit	$4.106b	$4.106b	-$9.95m	-$41.19m	$143.58m	$149.87m			$2.384b
5 Kraken	$3.619b	$3.619b	$68.88m	-$57.81m	$17.49m	-$559,975			$881.74m

- Dexes:


Total Value Locked
$55.705b
.csv
Change (24h) 2.26%
Lido Dominance 29.27%
Stablecoins Mcap $124.433b
Volume (24h) $2.337b
Total Funding Amount $96.934b
TVL
Volume
Raises
Stablecoins
Nov 1, 2023
TVL 55.705b USD
250b USD
200b USD
150b USD
100b USD
50b USD
0 USD
DeFiLlama
2019
2020
2021
2022
2023

Módulo VIII
API de Binance

8.1 Documentación

- Documentación
- Generación de API Key
- Métodos:
  - Publicos, no autenticados
  - Privados, autenticados
    - Cómo autenticar:
      - Tomar el body del request. Por ejemplo:

        “symbol=LTCBTC&side=BUY&type=LIMIT&timeInForce=GTC&quantity=1&price=0.1&recvWindow=5000&timestamp=1499827319559”
      - Firmar con HMAC-SHA256 usando el API SECRET como clave:

        openssl dgst -sha256 -hmac "$BINANCE_API_SECRET"
      - Enviar la firma como parámetro signature y el API KEY en el header X-MBX-APIKEY
      - Nunca poner el secret en código, slides o repos: si se filtra, revocarlo
    - Referencia

8.2 Métodos Públicos

- Son aquellos que involucran:
  - Obtener información pública
    - Precios
    - *Order book*
    - Información de los mercados disponibles
    - Mining
    - Staking
    - Funding rates
    - …

8.3 Métodos Privados

- Son todos aquellos que involucran:
  - Movimientos de fondos
  - Consulta de balances
  - *Trading*
  - Manejo de subcuentas
- Requieren autenticación

Módulo IX
Librería CCXT

9.1 ¿Qué es CCXT?

- Librería de Python que permite conectarse con múltiples exchanges
- API REST y websockets (gratis)
- Estandarización!

```python
exchange_id = 'binance'
exchange_class = getattr(ccxt, exchange_id)
exchange = exchange_class({
    'apiKey': 'YOUR_API_KEY',
    'secret': 'YOUR_SECRET',
})

hitbtc_markets = hitbtc.load_markets()

print(hitbtc.id, hitbtc_markets)
print(bitmex.id, bitmex.load_markets())
print(huobipro.id, huobipro.load_markets())

print(hitbtc.fetch_order_book(hitbtc.symbols[0]))
print(bitmex.fetch_ticker('BTC/USD'))
print(huobipro.fetch_trades('LTC/USDT'))

print(exmo.fetch_balance())

# sell one ฿ for market price and receive $ right now
print(exmo.id, exmo.create_market_sell_order('BTC/USD', 1))

# limit buy BTC/EUR, you pay €2500 and receive ฿1  when the order is closed
print(exmo.id, exmo.create_limit_buy_order('BTC/EUR', 1, 2500.00))

# pass/redefine custom exchange-specific order params: type, amount, price, flags, etc...
kraken.create_market_buy_order('BTC/USD', 1, {'trading_agreement': 'agree'})
```

Anexo
docker compose: un stack blockchain local

A.1 docker compose: varios containers juntos

- Un archivo (docker-compose.yml) describe varios servicios, cómo se conectan y en qué orden arrancan

```
anvil      blockchain EVM local: un bloque cada 2 s, 10 cuentas con ETH de prueba
deployer   deploya QuantToken (el contrato de Remix) y hace 3 transfers
postgres   base de datos
indexer    guarda cada bloque y cada evento Transfer en Postgres
chain-api  la imagen de la clase 4, apuntando a anvil
adminer    interfaz web para mirar la base
```

- Adentro de la red de compose los servicios se encuentran por nombre: http://anvil:8545
- ¿Por qué una chain local? Transacciones sin wallet de testnet, sin faucet y sin esperar

A.2 El docker-compose.yml

```
services:
  anvil:
    image: ghcr.io/foundry-rs/foundry:v1.5.1
    command: ["anvil --host 0.0.0.0 --block-time 2"]
    ports: ["8546:8545"]
    healthcheck:
      test: ["CMD", "cast", "block-number", "--rpc-url", "http://localhost:8545"]
  indexer:
    build: ./indexer
    environment:
      WS_URL: ws://anvil:8545
      DATABASE_URL: postgresql://quant:quant@postgres:5432/chain
    depends_on:
      anvil:    {condition: service_healthy}
      postgres: {condition: service_healthy}
```

- depends_on + healthcheck: nadie arranca antes de que la chain y la base respondan

A.3 Levantar el stack y mandar transacciones

```
cd modulo-2/clase_2/docker
docker compose up -d --build
docker compose ps -a
docker compose logs deployer          # dirección del token y transfers
docker compose logs -f indexer        # cada bloque en vivo
```

- Consultar la chain local con la chain-api (puerto 8001):

```
curl localhost:8001/health            # chain_id 31337 = anvil
curl "localhost:8001/token/0x5FbDB2315678afecb367f032d93F642f64180aa3?holder=..."
```

- Mandar una transfer y verla llegar al indexer:

```
docker compose run --rm deployer python transfer.py --to 2 --amount 3
```

A.4 El indexer: de eventos a SQL

- Cada Transfer(from, to, value) del ERC-20 termina en una tabla:

```
docker compose exec postgres psql -U quant -d chain
SELECT block_number, from_address, to_address, round(value / 1e18, 4)
FROM transfers ORDER BY block_number;
SELECT address, round(balance / 1e18, 4) FROM balances;
```

- Los saldos salen solo de los eventos y coinciden con balanceOf: así funcionan los exploradores
- Web: http://localhost:8088 (Adminer, usuario y contraseña quant)
- Romperlo a propósito: docker compose restart anvil
  - La chain vuelve a cero: el indexer detecta que su último bloque ya no existe (como un reorg) y re-indexa
- Apagar: docker compose down (-v para borrar también la base)

Hands on: *docker compose*

- Levantar el stack: docker compose up -d --build
- Leer los logs del deployer: ¿en qué dirección quedó el token? ¿Por qué es siempre la misma?
- Mandar 3 transfers con transfer.py y verlas llegar en docker compose logs -f indexer
- Con SQL: ¿qué cuenta recibió más QNT? ¿Coincide con /token/...?holder= de la chain-api?
- Reiniciar anvil y explicar qué hace el indexer (¿qué tiene que ver con un reorg?)
- Punto de partida para la tarea: indexar eventos Swap en vez de Transfer
- Instrucciones completas: modulo-2/clase_2/docker/INSTRUCCIONES.md

Conclusión y resumen

Conclusión y resumen

- Módulo V: Librería asyncio para Python
  - Comentario rápido, ventajas de asyncio
- Módulo VI: Discusión de la tarea propuesta
  - Implementación de la interfaz
  - Muchos exchanges son forks de uni! es un buen lugar para empezar
- Modulo VII: Repaso de mercados centralizados
  - Gran tamaño, gran liquidez, gran velocidad. Menos privacidad
- Modulo VIII: API de Binance
  - Integración, métodos públicos y privados
- Módulo IX: Librería CCXT
  - Definición
  - Implementación interfaz
  - Ejemplos!
- Anexo: docker compose
  - Varios containers juntos: chain local, token, indexer y Postgres
  - De eventos on-chain a SQL

Tarea propuesta

Tarea propuesta

- Desarrollar un servicio que, ante cada evento de swap generado por un par determinado (por ejemplo, WPOL/USDC; WPOL es el ex-WMATIC), muestre el precio actual del par en Sushiswap, Quickswap, Binance, y OKX.
- Guía:
  - Para obtener la dirección del pool y de los tokens:
    - Crear el contrato factory de UNIV2 para Sushiswap y QuickSwap
    - Usar el método getPair con el address de cada uno de los tokens
    - Para obtener el address de los tokens:
      - En sushiswap, ir a la sección de pools
      - Filtrar por polygon
      - Buscar el pool WPOL/USDC (ex-wMATIC), y ahí está la información
  - Después, crear un filtro de web3 con el evento de swap y los addresses del par de los dos protocolos
  - Finalmente, en cada evento, imprimir los precios de los 4 exchanges

